IY4113 Milestone 1 Part 2

Assessment Details	Please Complete All Details
Group	B
Module Title	Applied Software Engineering using Object-Orientated Programming
Assessment Type	Java programming with inheritance and file handling
Module Tutor Name	Jonathan Shore
Student ID Number	P0460817
Date of Submission	22/03/2026
Word Count	524
GitHub Link	GitHub - BatuhanSert777/IY4113-Milestone: IY4113 Milestone 1 – CityRide Lite planning and documentation. · GitHub

- [x] *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*
- [x] *Where I have used AI, I have cited and referenced appropriately.

Purpose of the Program

The purpose of this program is to extend the original **CityRide Lite** application from Part 1 into a more realistic transport fare companion system. In Part 2, the program must support both **Rider** and **Admin** roles, use **inheritance** in its object-oriented design, and handle **persistent storage** through **JSON** and **CSV** files. For a rider, system should allow the user to create or load a profile, store details, name, passenger type and default payment option and manage journeys for the current day. The rider must be able to add, edit and delete journeys, import journeys from a CSV file, export current journeys to CSV, and generate an end-of-day summary. The system

must calculate fares using the same core fare rules from Part 1, including base fares, discounts and daily caps. For an administrator, the system must provide a password-protected menu to manage the active fare configuration. This includes viewing, adding, updating and deleting base fares, passenger discounts, daily caps and peak time windows. Any updates made by the admin must be validated before they are saved. Overall, the program is designed to act as a menu-driven Java console application that gives riders accurate journey cost tracking and summaries, while also allowing administrators to maintain the configuration that controls the fare system.

Core Program Functionality

- Support two roles: Rider and Admin
- Load configuration data when the program starts
- Use safe default values if the configuration file is missing
- Create, load and save rider profiles using JSON
- Add, edit and delete journeys for the active day
- Import journeys from CSV and export current journeys to CSV
- Calculate fares using zones, time band, passenger type, discounts and daily caps
- Display running totals and end-of-day summaries
- Export summaries as CSV and human-readable text reports
- Allow admins to manage fare rules through a password-protected menu
- Validate all user input before saving data

System Constraints

- The application is a Java console program and will be menu-driven rather than graphical.
- The program must use object-oriented design and include inheritance.
- The dataset file `CityRideDataset.java` contains rules and constraints that the fare logic must follow.
- The program must use JSON for configuration and rider profile data.
- The program must use CSV for journey imports/exports and summary reports.
- Invalid data must not be saved to file.
- All prompts should clearly show the expected format and example values.
- The solution should follow the NTIC Guide to Good Programming, including clear naming, functional decomposition, private fields where appropriate and readable code structure.

Input Process Output Table

Input Process Output Table

Planned Part 2 system based on the assignment brief and current CityRide Lite design

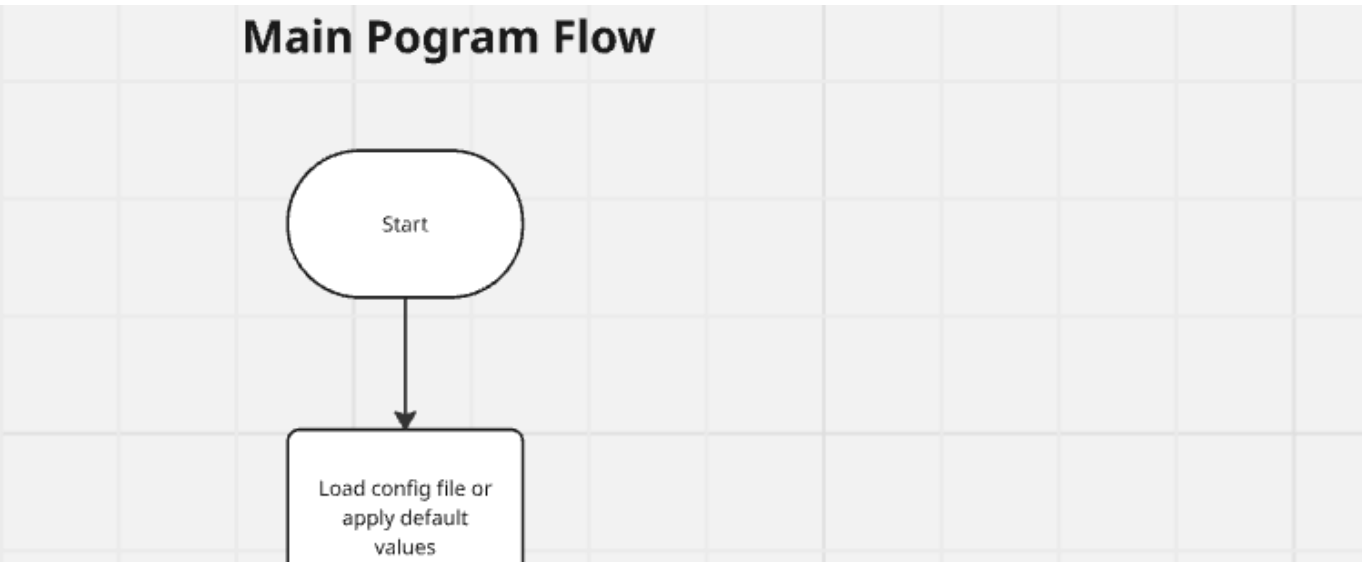
Feature	Inputs	Processing	Outputs
Start program and choose role	User launches the program and selects Rider or Admin	System loads the config file; if it is missing, safe default values are applied; the selected role is checked and the user is routed to the correct menu	Rider menu displayed, Admin login prompt displayed, or invalid choice message
Create rider profile	User enters rider name, passenger type, and default payment option	System displays expected format/examples, validates the entered values, and creates a rider profile object in memory	Profile created confirmation or validation error message
Load rider profile	User selects load profile and enters or chooses a profile file	System reads rider profile data from a JSON file, checks that the file exists, and validates the loaded data before use	Rider profile displayed or file/validation error message
Save rider profile	User selects save profile	System converts the current rider profile into JSON format, validates the data, and saves it only if valid	Save confirmation message, or validation/file saving error message with no file saved
Add journey	User enters date/time, from zone, to zone, time band, and passenger type	System displays expected format/examples, validates all inputs, calculates zones crossed, looks up the correct fare, applies discount and daily cap, generates a unique journey ID, and stores the journey	Journey added confirmation showing journey ID and charged fare, or validation error message
Edit journey	User enters a journey ID and updated journey details	System checks that the journey ID exists, validates the new values, updates the journey, and recalculates fares and totals where required	Updated journey details displayed or invalid ID/validation error message
Delete journey	User enters a journey ID and confirms deletion	System checks that the journey exists, removes it, and recalculates fares and totals after deletion	Journey deleted confirmation or journey not found message
List journeys	User selects the list journeys option	System retrieves all stored journeys for the active rider and formats them for display	Full journey list displayed or no journeys recorded message
Filter journeys	User selects a filter type and enters a filter value	System compares stored journeys against the selected filter, such as passenger type, time band, date, from zone, or to zone, and collects matching records	Filtered journey list displayed or no matching journeys found message
Import journeys from CSV	User selects import journeys and enters a CSV file name or path	System reads the CSV rows, validates each row, converts valid rows into journey objects, and adds them to the rider's journey list; invalid data is not saved	Imported journeys displayed with success message, or CSV format/validation error message
Export journeys to CSV	User selects export journeys and enters an export file name	System converts the current journey list into CSV format and writes it to an output file	CSV export success message or file writing error message

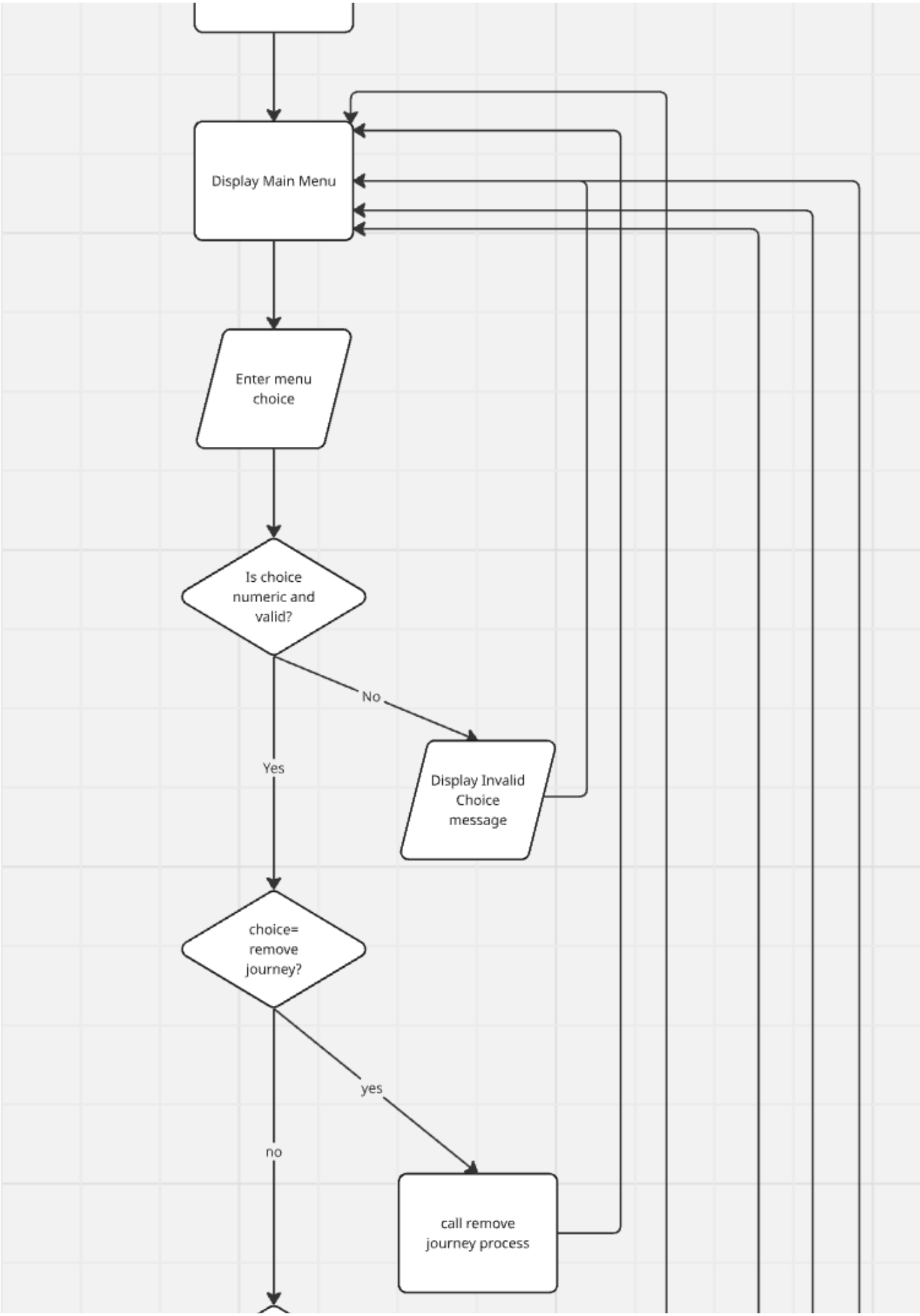
Feature	Inputs	Processing	Outputs
Calculate fare and running total	User enters from zone, to zone, time band, and passenger type	System uses the active config and current totals to determine the base fare, apply passenger discount, check daily cap status, calculate the charged fare, and update the running total	Base fare, discounted fare, charged fare, updated total, and cap status displayed
Display daily summary	User selects the summary option	System uses the stored journeys to calculate number of journeys, total cost, average cost, most expensive journey ID, cap reached status, savings compared to uncapped fares, peak/off-peak counts, and zone statistics	End-of-day summary displayed on screen
Export summary reports	User selects export summary and enters an export file name	System uses the current summary, rider name, and date to generate a CSV line-item report and a human-readable text report saved with rider name and date	CSV report and text summary saved, or file writing error message
Admin login	User enters admin password	System validates the password before allowing access to admin functions	Admin menu displayed or incorrect password message
View, add, update and delete config values	Admin selects a config action and enters fare, discount, cap, or peak window values	System validates entered config values, updates the config object, and saves to JSON only if valid	Updated config displayed with success message, or validation error message with no file saved
Exit program and save state	User selects exit and chooses whether to save current data	System offers a save option, then saves the current rider profile and journeys if confirmed before closing the program	Save confirmation and exit message, or exit without saving message

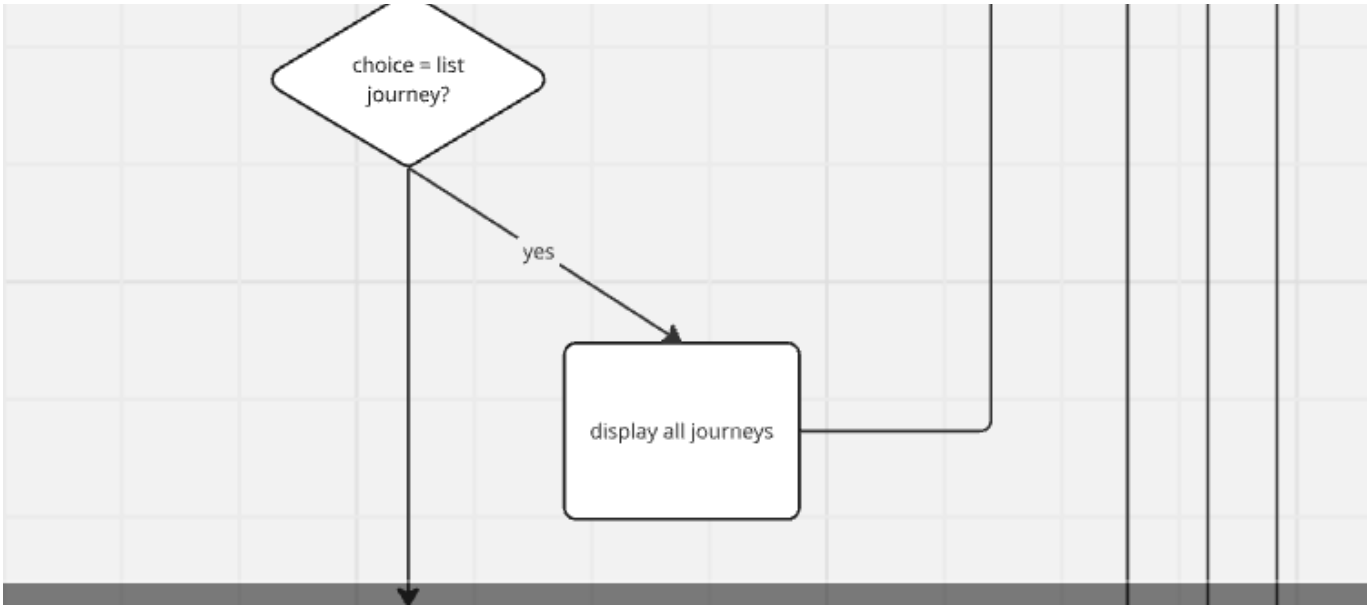
Algorithm Design

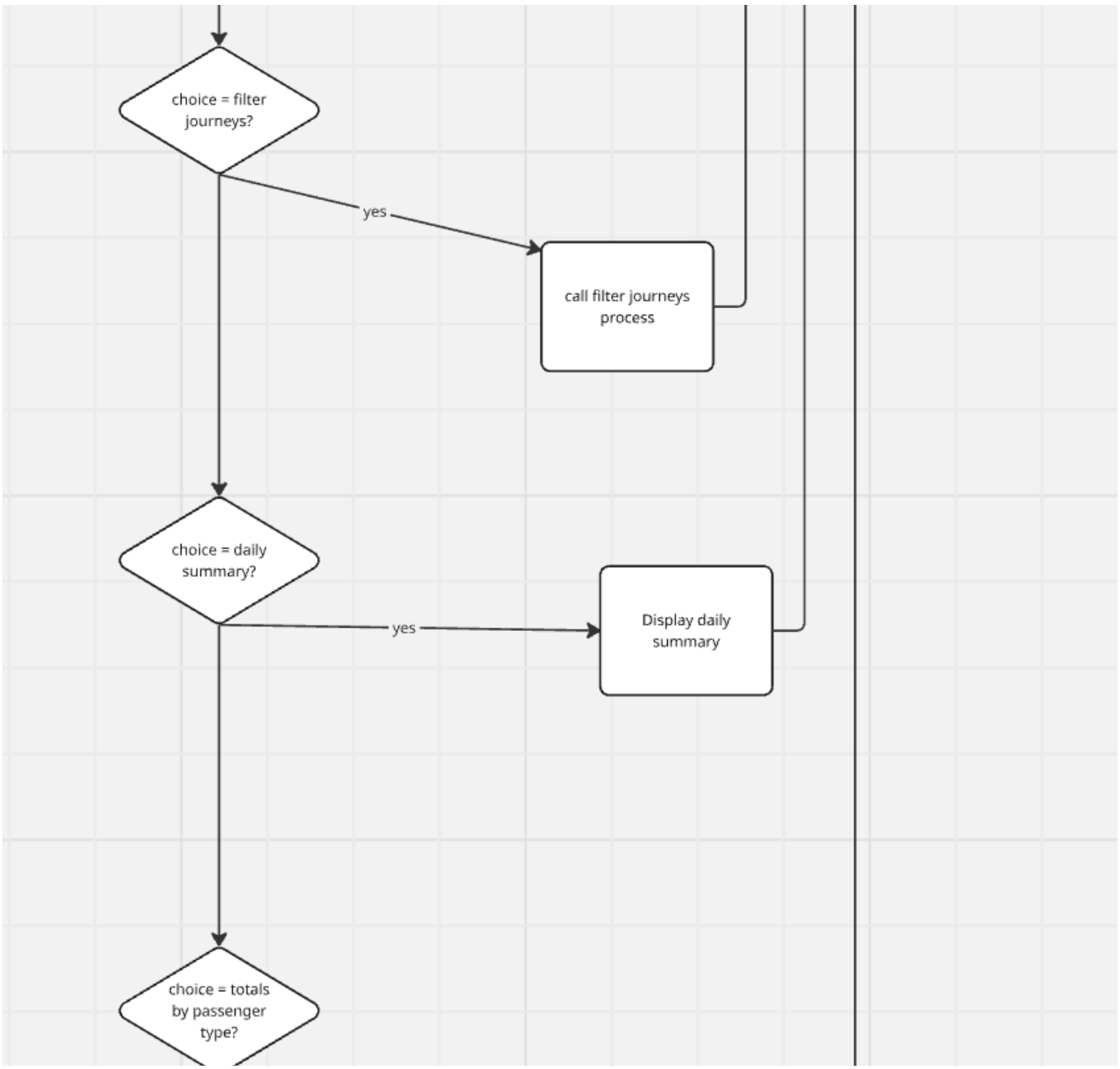
The algorithm design is shown using two flowcharts. The first flowchart shows the overall program flow, while the second shows the add journey process in more detail. A UML class diagram is also included to show the structure of the proposed system.

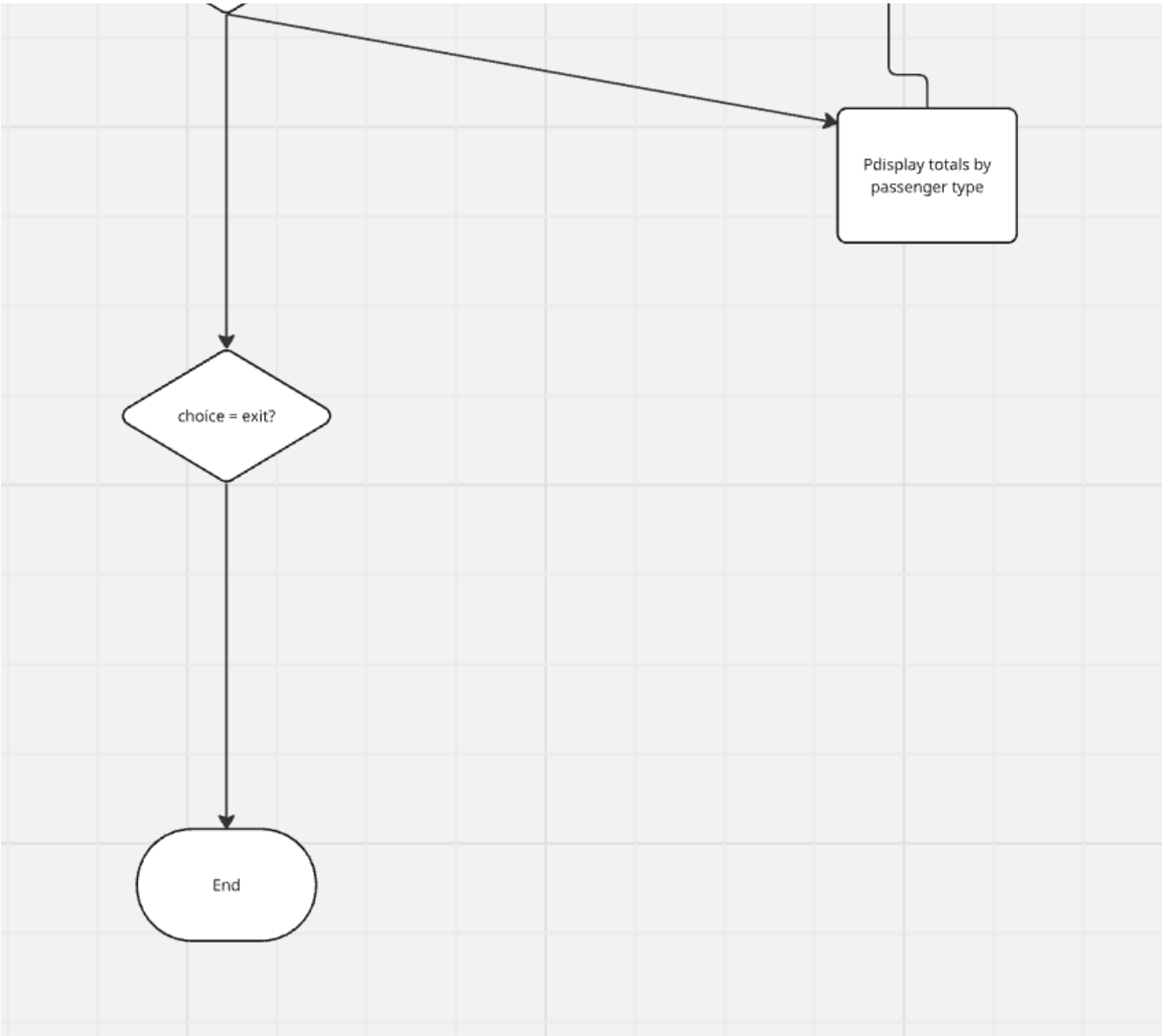
Main Program Flowchart



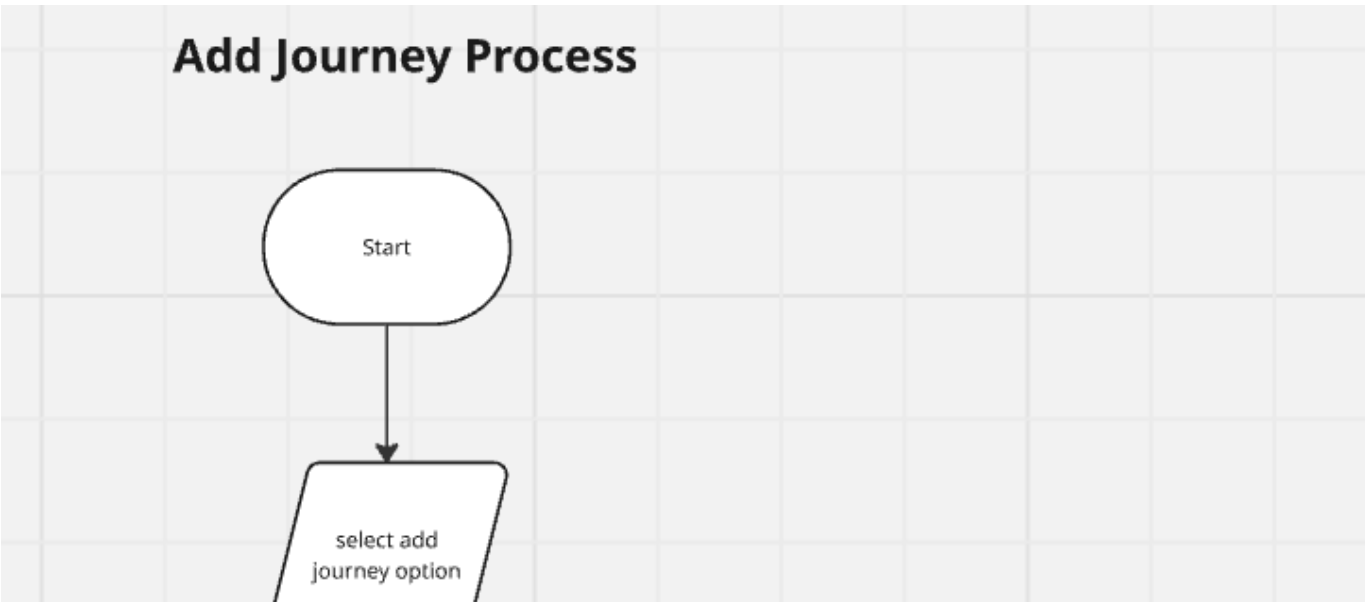

Display Main Menu
Enter menu choice
Is choice numeric and valid?
No
Display Invalid Choice message
Yes
choice= remove journey?
yes
call remove journey process
no

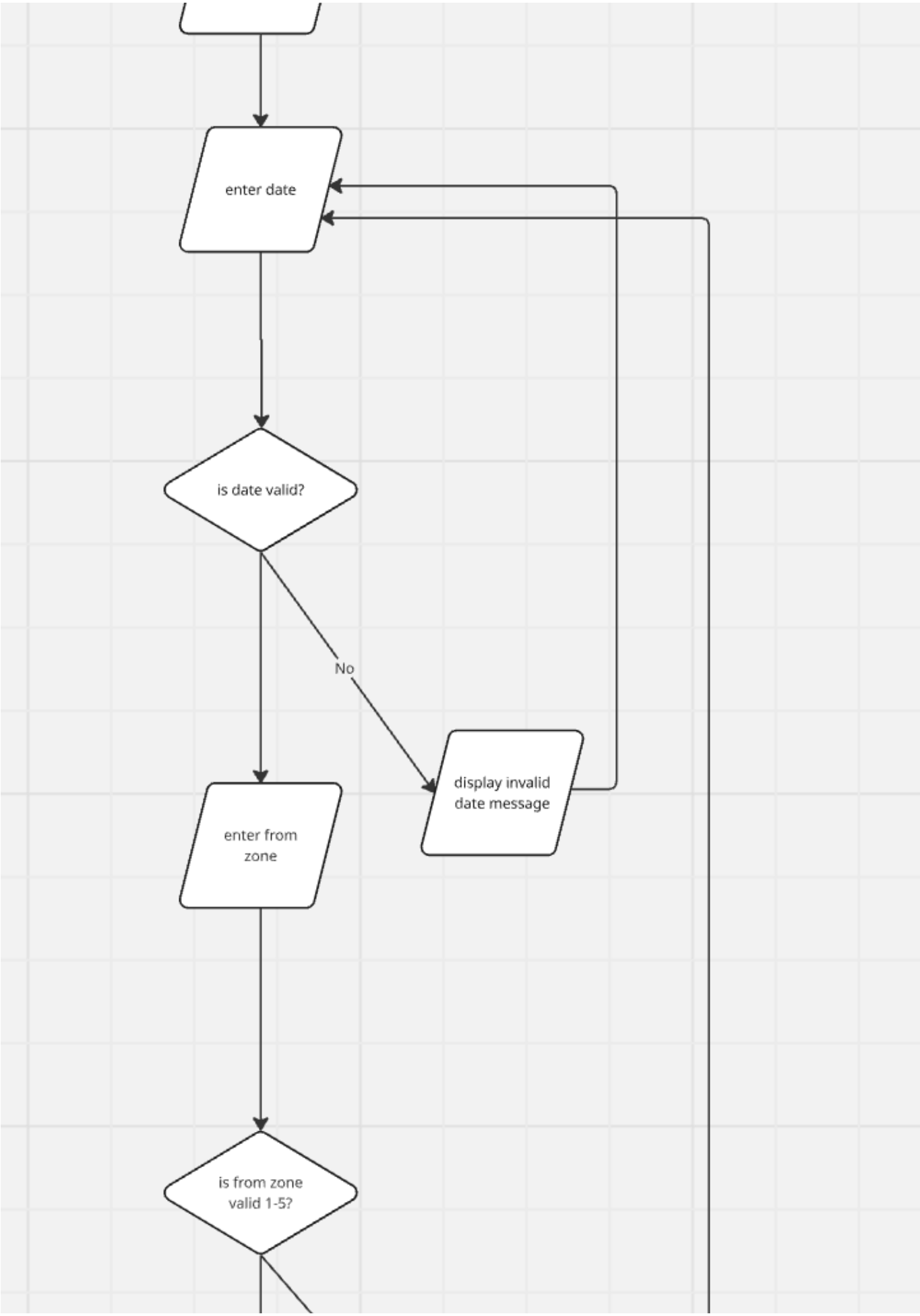

choice = list journey?
yes
display all journeys

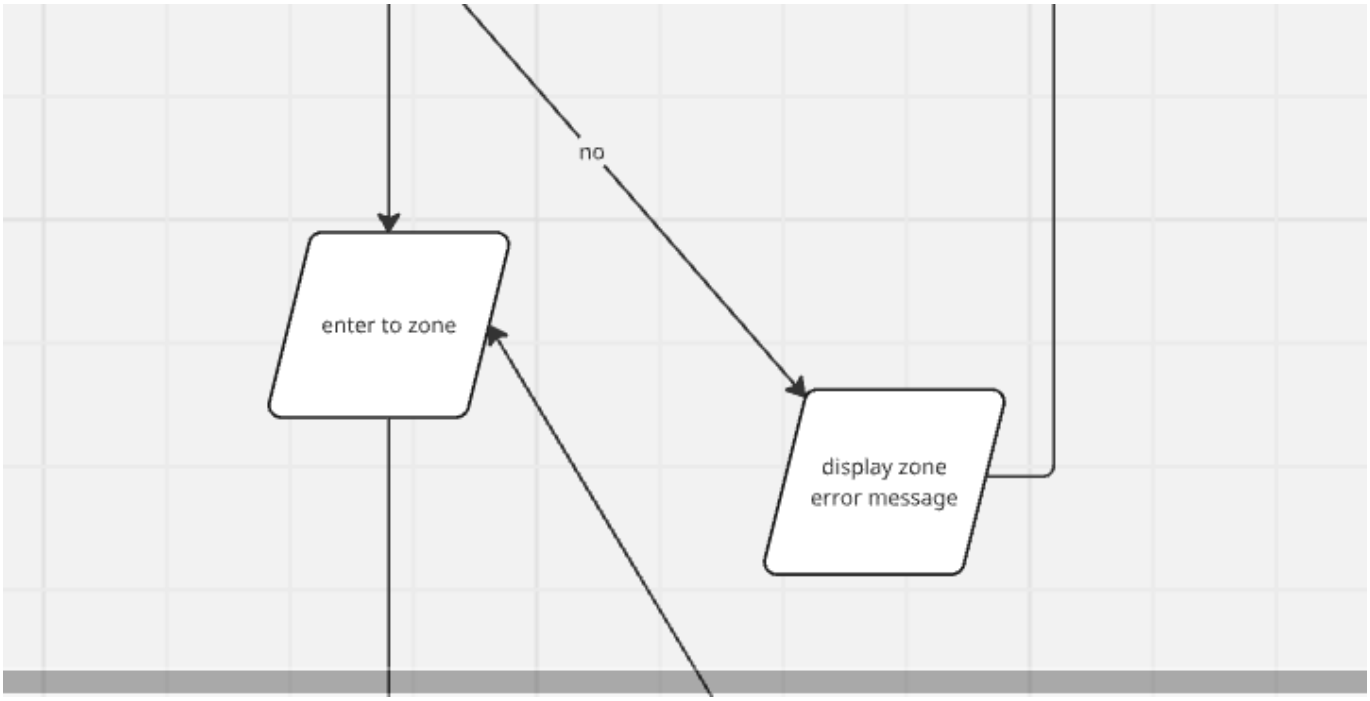

choice = filter journeys?
yes
call filter journeys process
choice = daily summary?
yes
Display daily summary
choice = totals by passenger type?

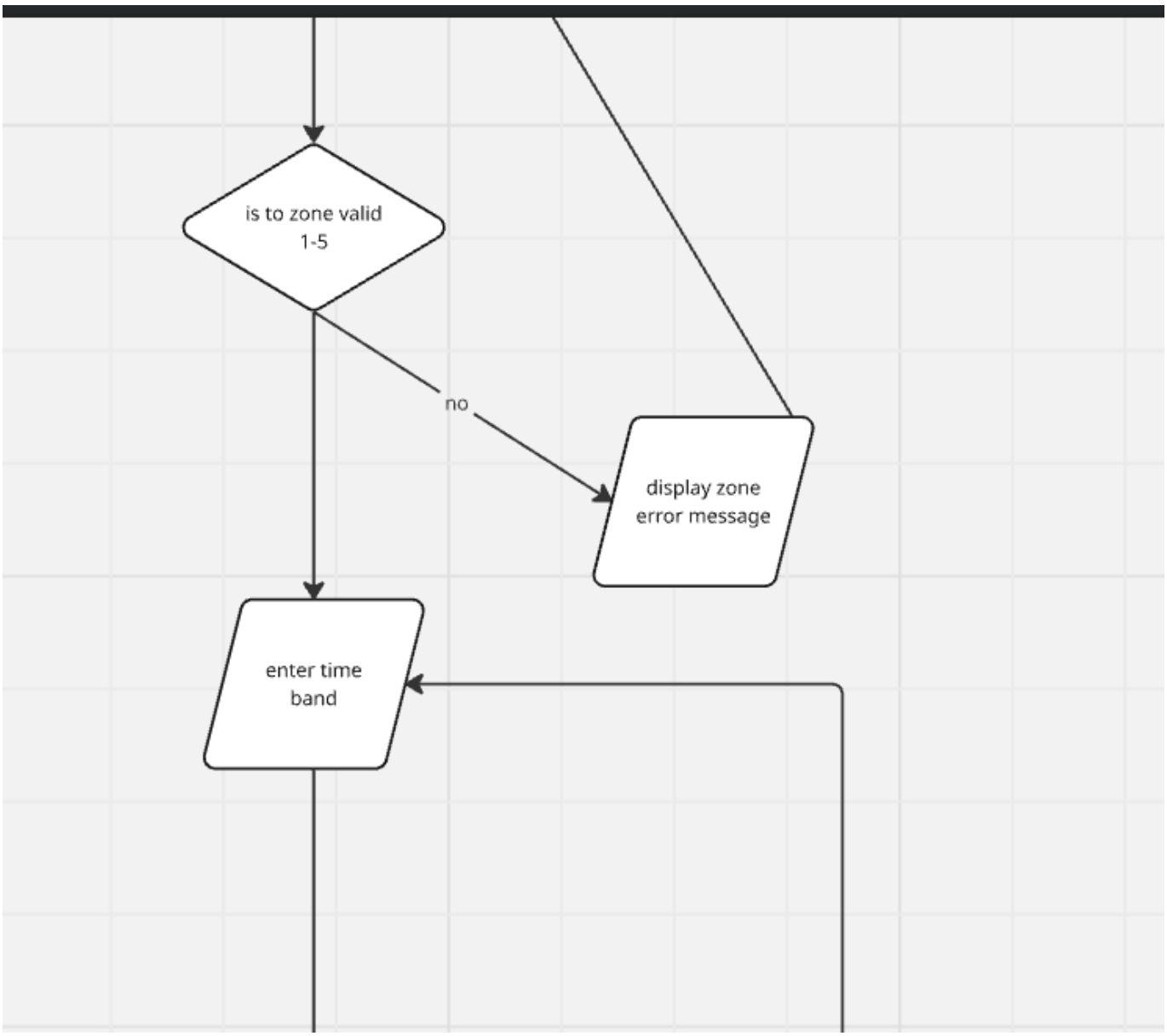


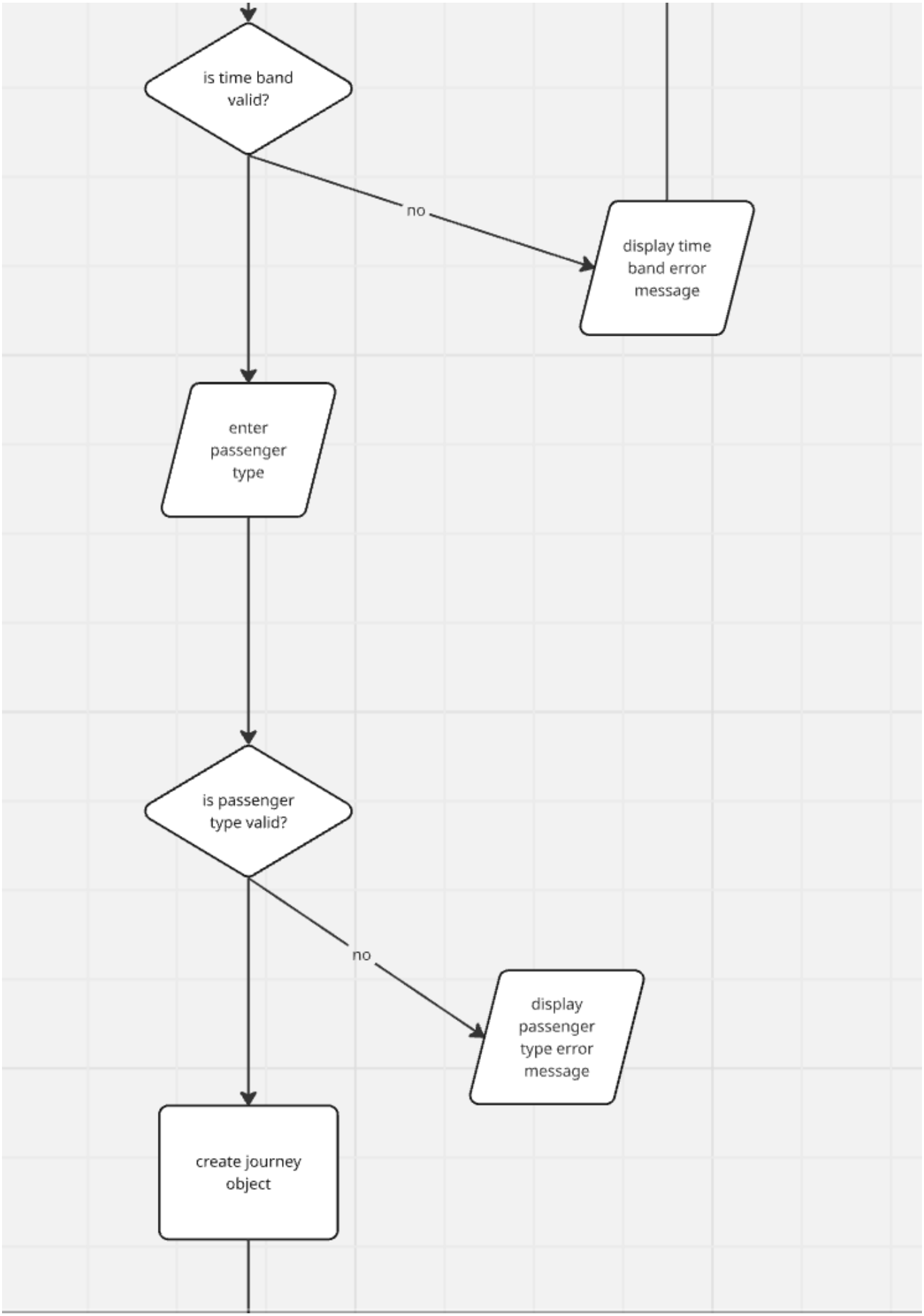
Add Journey Flowchart

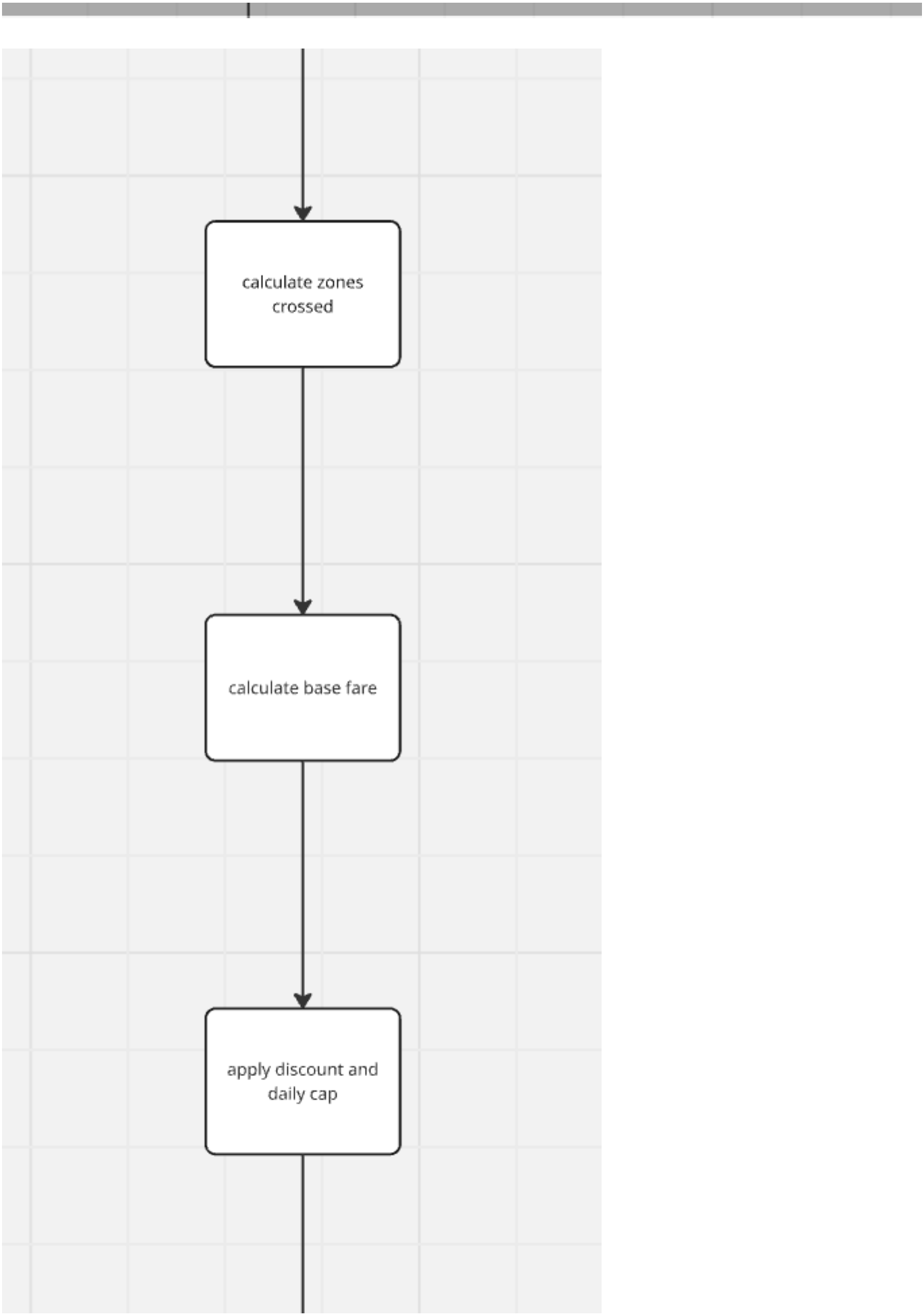


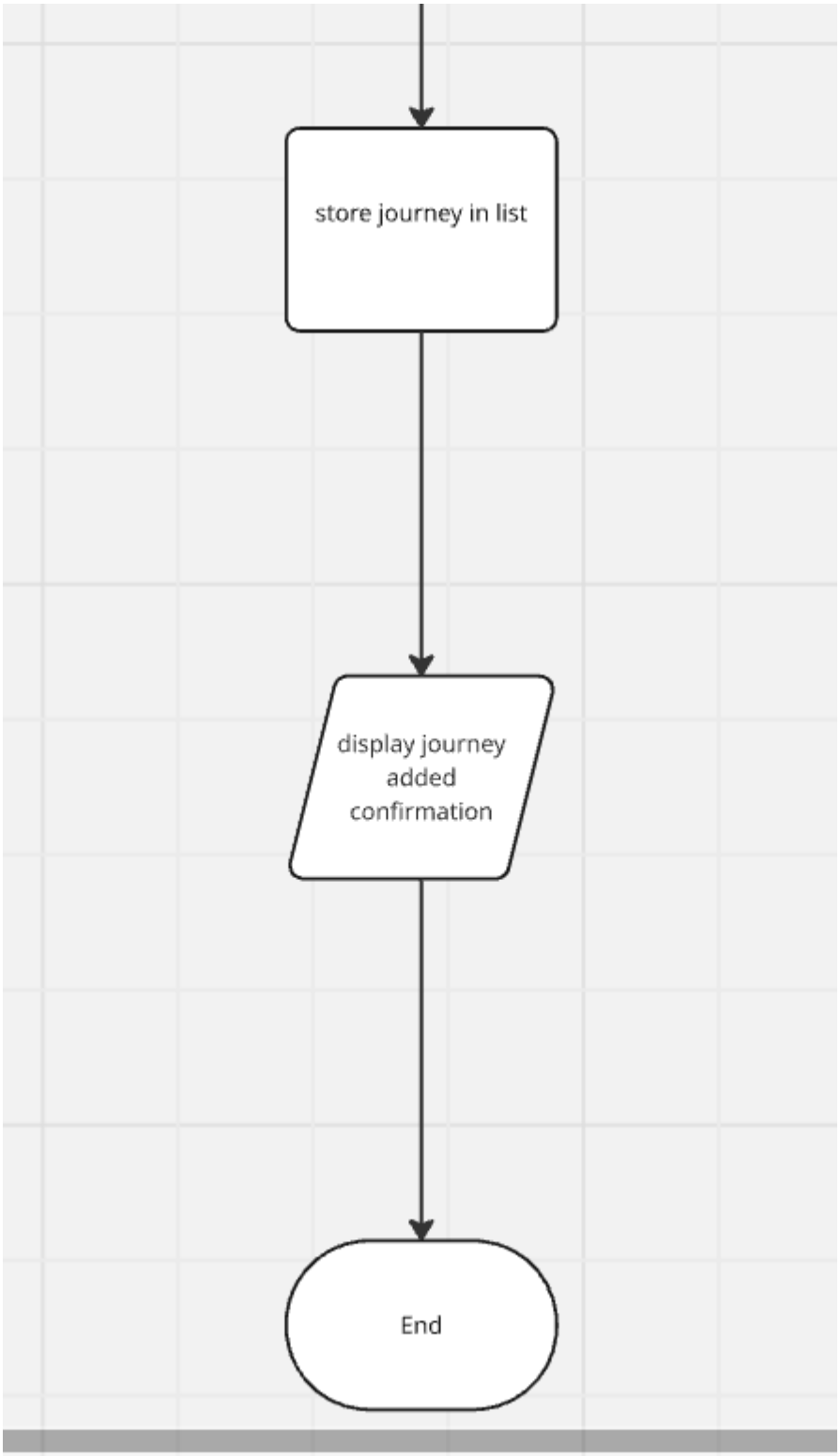

enter date
is date valid?
No
display invalid date message
enter from zone
is from zone valid 1-5?

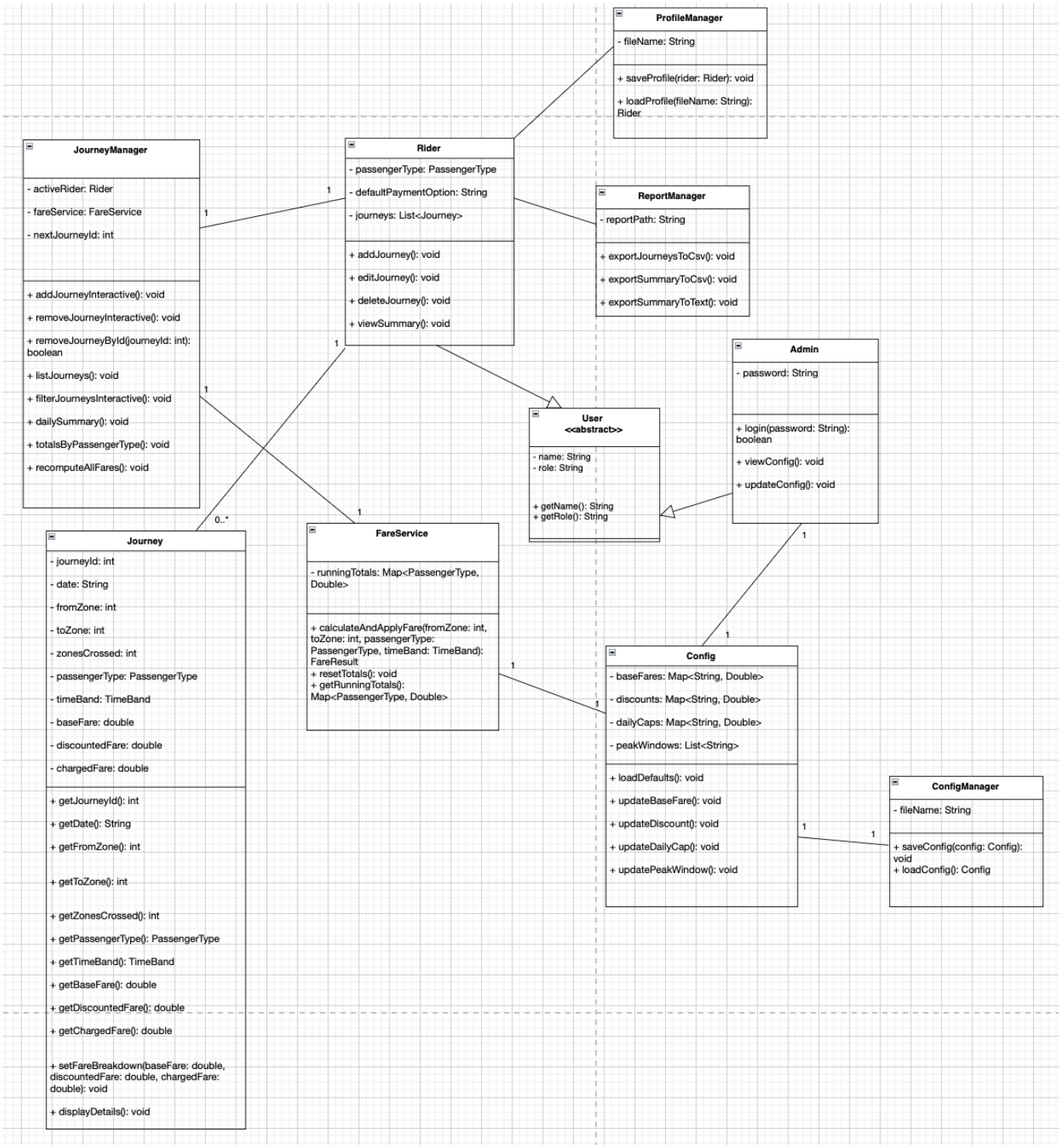

no
enter to zone
display zone
error message


is to zone valid
1-5
no
display zone
error message
enter time
band


is time band valid?
no
display time band error message
enter passenger type
is passenger type valid?
no
display passenger type error message
create journey object


calculate zones crossed
calculate base fare
apply discount and daily cap


store journey in list
display journey added confirmation
End



Research

Research Example 1

Name of program: Citymapper

Reference (link): https://citymapper.com/

What it does well :

- It compares different travel options in real time across multiple transport modes.
- It provides turn-by-turn navigation, which makes journeys easier to follow.
- It presents route and travel information in a clear and user-friendly way.

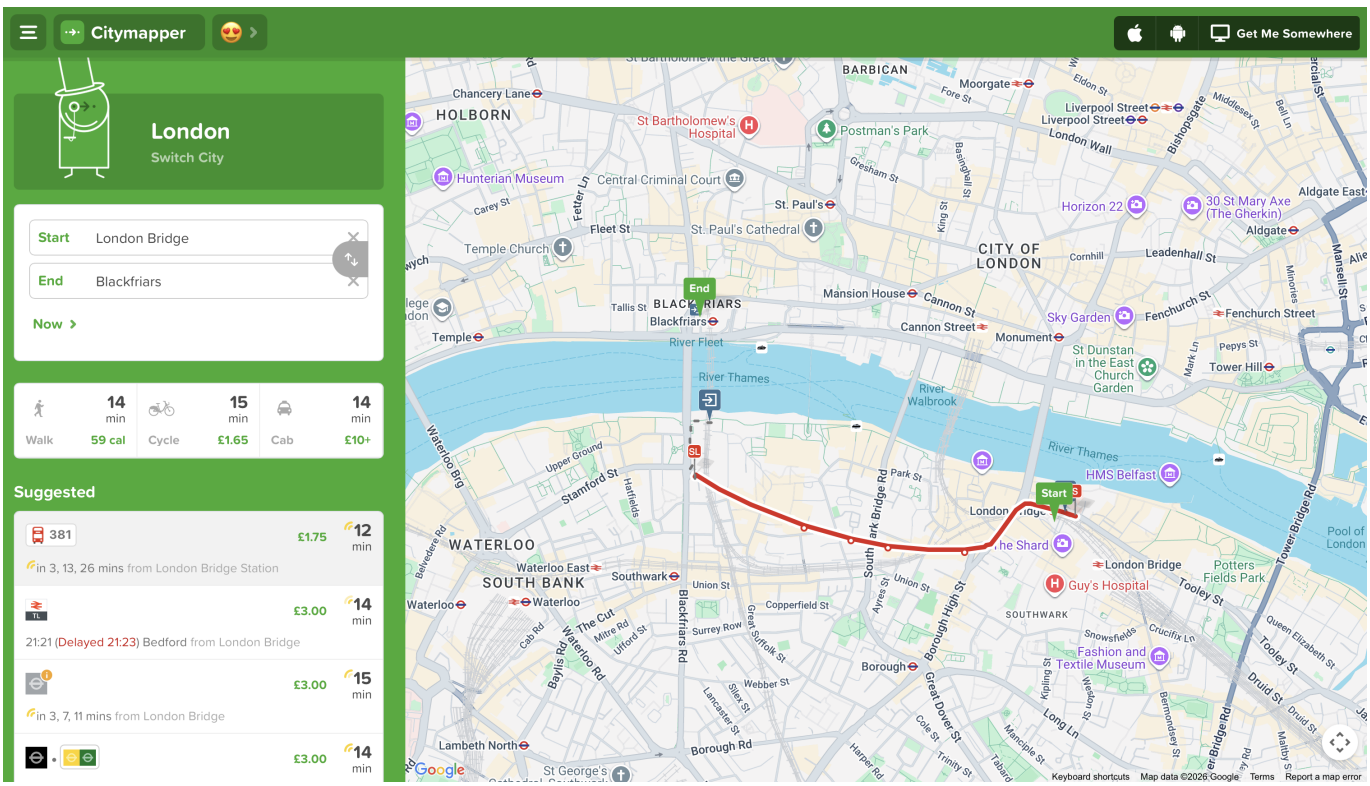
What it does poorly :

- It focuses more on journey planning than on detailed fare rule management or administrative control.
- It includes many advanced features, which can make it feel more complex than a simple coursework system.

Key design ideas you could reuse :

- Clear menu-based journey selection.
- Easy-to-read output showing route and cost information.
- A practical focus on helping the user make transport decisions quickly.

Screenshot :



Research Example 2

Name of program: TfL Go

Reference (link): https://tfl.gov.uk/maps_/tfl-go

What it does well :

- It helps users plan journeys and manage their travel costs.
- It provides clear public transport information in a focused and practical way.
- It supports accessibility features such as step-free journey planning.

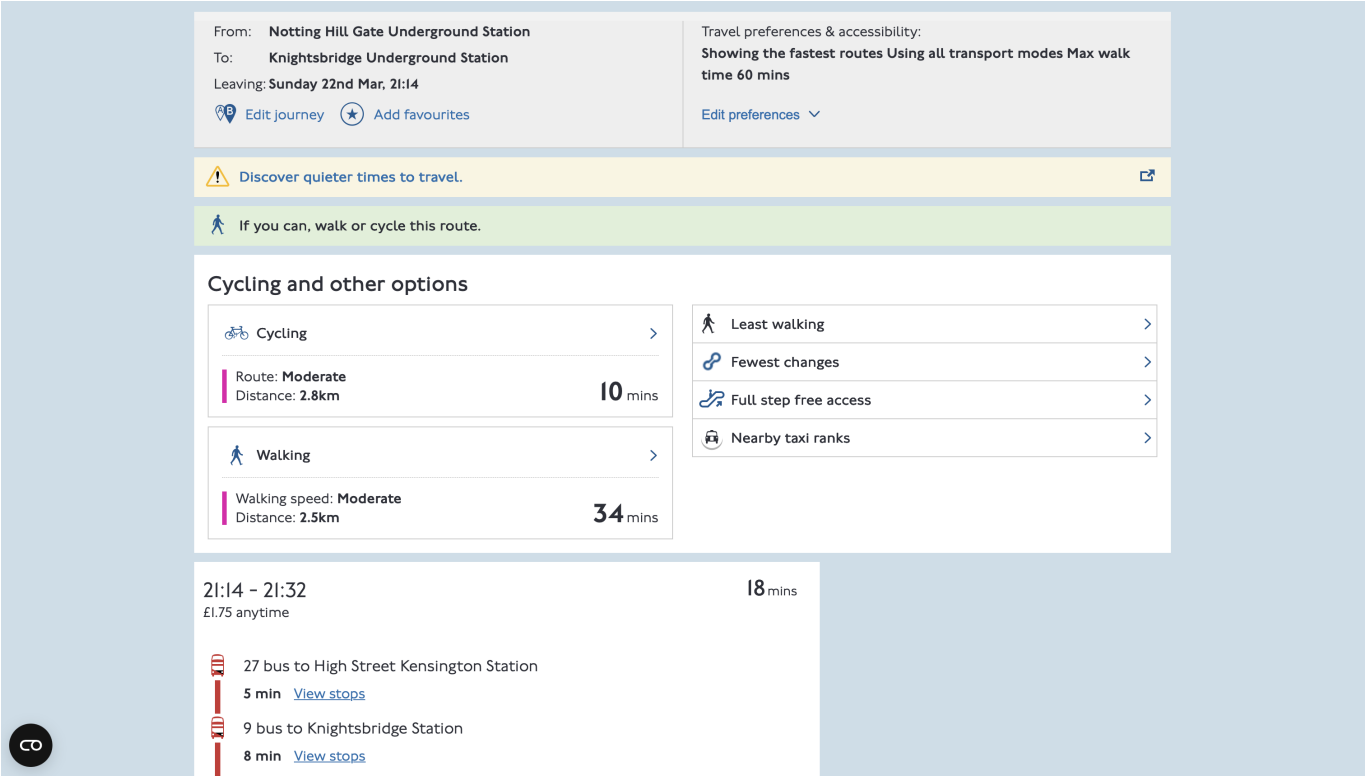
What it does poorly :

- It is mainly designed for the London transport network, so it is less flexible than a general transport fare system.
- It does not provide user-facing admin controls for changing fare rules or system configuration.

Key design ideas you could reuse :

- Clear separation between journey planning and travel cost information.
- Simple user interaction with direct outputs after each input.
- Practical use of summaries and travel information for everyday users.

Screenshot :



How the research informed my design

The research helped me understand how real transport-related systems present journey and fare information clearly to users. Citymapper influenced the idea of showing journey options and travel information in a simple and readable way. TfL Go was useful because it combines journey planning with travel cost support, which is closer to the aim of CityRide Lite Part 2. From these examples, I decided that my own system should have a clear menu structure, readable outputs, and practical journey summaries. I also decided to keep normal rider features separate from admin configuration features so that the program remains organised and easy to use.

Gantt Chart

IY4113 Part 2 Milestone 1 - Gantt Chart									
Task	Start Date	End Date	Duration (days)	17-Mar	18-Mar	19-Mar	20-Mar	21-Mar	22-Mar
Analyse Part 2 brief and submission requirements	17/03/2026	17/03/2026	1	■					
Identify core functions and system constraints	17/03/2026	18/03/2026	2	■	■				
Create Input Process Output table	18/03/2026	20/03/2026	3		■	■	■		
Research similar transport applications	19/03/2026	21/03/2026	3			■	■	■	
Design UML class diagram	20/03/2026	22/03/2026	3				■	■	■
Create main program flowchart	21/03/2026	22/03/2026	2					■	■
Create add journey flowchart	21/03/2026	22/03/2026	2					■	■
Prepare research summary and screenshots	21/03/2026	22/03/2026	2					■	■
Write diary entries and review milestone content	22/03/2026	22/03/2026	1						■
Final proofreading, PDF export, and submission	22/03/2026	22/03/2026	1						■

Diary Entries

Diary Entry 1 – Analysing the brief

I started by reading the Part 2 assignment brief carefully so that I could understand the new requirements and avoid repeating mistakes from Part 1. I identified that the most important changes are inheritance, file handling, rider and admin roles, and the use of JSON and CSV files. I also reviewed previous feedback to remind myself of the areas that lost marks before, especially IPO accuracy, diagram quality, and lack of detail. The main difficulty at this stage was deciding how to plan the full Part 2 system even though my current implementation is still more limited. I solved this by treating the brief as the source of truth and planning the complete system before thinking about coding.

Diary Entry 2 – Planning the system

After analysing the brief, I focused on planning the structure of the proposed program. I wrote the purpose of the program, identified the core functionality, and listed the key system constraints. I then created the IPO table for the main functions of the planned Part 2 system. This time I paid much more attention to making sure each function had a real input, a clear process, and an actual output, because that was one of the weaknesses identified in my previous feedback. The main challenge was balancing my current CityRide Lite implementation with the wider Part 2 requirements. To deal with this, I presented the IPO as a planned Part 2 system built on my existing design.

Diary Entry 3 – Research and design decisions

I researched existing transport-related systems to understand how similar programs present journey and fare information to users. I used Citymapper and TfL Go as examples because both show journey planning and travel information clearly, even though they are larger systems than my own coursework project. This research helped me think more carefully about menu structure, readability of outputs, and how to separate rider functions from admin functions. I used these ideas to improve my own planned design, especially in relation to journey summaries, user interaction, and keeping the system organised. The main difficulty was selecting design ideas that were realistic for a Level 1 Java console application, so I focused only on features that could be simplified and adapted to my own project.

Diary Entry 4 – Creating the algorithm and class design

I then developed the design documentation for the program by creating flowcharts and a UML class diagram. I used the flowcharts to show the overall structure of the program and one of the main sub-processes in more detail. For the class diagram, I designed a structure that reflects both my current implementation and the planned Part 2 extensions, especially the use of inheritance through User, Rider, and Admin. I also included manager classes such as JourneyManager, ProfileManager, ConfigManager, and ReportManager so that the responsibilities of the system are shown more clearly. A difficulty here was making sure the diagrams stayed readable and used acceptable notation. I improved this by simplifying the relationships, using correct inheritance arrows, and avoiding unnecessary links.

Diary Entry 5 – Reviewing and refining the milestone

Finally, I reviewed the milestone as a whole and checked whether each section matched the assignment brief. I refined the wording of the planning sections, improved the research explanations, and adjusted the diagrams so that they better represent the intended Part 2 system. I also made sure that my work shows stronger links between the brief, my planning decisions, and the design documentation than in Part 1. The main issue at this final stage was time management, because there were several sections to complete and refine before submission. To manage this, I prioritised the sections that are directly mentioned in the brief and focused on making each section clear, relevant, and consistent with the planned system.